Jio FAQ Chatbot — Functionalities & Implementation

Table of Contents

---

1. Text Chat

What it does: The core feature. A user sends a text message and gets a contextual response — either a Jio FAQ answer or a general reply.

How it's implemented:

1. `POST /api/chat` in `server.py` receives the message with a JWT token and optional `session_id`.
2. If the session doesn't already exist in the `chat_sessions` Supabase table, it gets auto-created. The title is generated by Groq Llama 3.1 8B from the first message.
3. The message is passed into the **LangGraph workflow** (`app.py`) as the initial state. The state type is defined in `agent_state.py` as `GraphState`.
4. The workflow has three nodes: `retrieve_node` → conditional routing → `generate_node` or `general_generation_node`.
5. The final `answer` from the state is returned in the API response.
6. Two background tasks fire asynchronously after each response:
7. `evaluate_and_save_memory_bg` — evaluates if anything should be saved to long-term memory.
8. `summarize_short_term_memory_bg` — trims the in-session conversation history if it grows beyond 5 messages.

---

2. Audio / Voice Chat

The project supports two distinct audio modes: **file-based upload** and **real-time WebSocket streaming**.

2a. File-Based Audio Chat

Endpoint: `POST /api/chat/audio`

1. Frontend records audio via the browser's `MediaRecorder` API and sends the raw bytes as a file upload.
2. `server.py` runs `transcribe_audio_file()` from `get_transcript.py`. This uses `ffmpeg` to normalize audio to 16kHz mono WAV, then calls Sarvam AI's REST STT API (`saaras:v3`).
3. The resulting transcript goes through the same LangGraph workflow as text chat.
4. The answer text is passed to `generate_speech()` in `get_audio.py`. It splits the text into sentences, fires parallel `async` calls to Sarvam TTS (`bulbul:v3`), concatenates the WAV chunks, and returns a base64-encoded audio string.
5. The response contains both the text answer and `audio_base64` for playback.

2b. Real-Time Audio Streaming (WebSocket)

Endpoint: `WS /api/audio_stream/ws`

1. The frontend connects via WebSocket and sends an `auth` message with the JWT.
2. Raw 16kHz mono PCM audio chunks are streamed from the browser (encoded as base64).

3. On the server, **Silero VAD** (`VADIterator` from `get_transcript.py`) processes 512-sample windows to detect speech start and end.
4. On speech end, the buffered PCM is passed to `transcribe_pcm()` which wraps it in a WAV header in-memory and sends it to Sarvam STT.
5. The transcript triggers the LangGraph RAG pipeline. The LLM response is streamed back token-by-token as `text_chunk` WebSocket messages.
6. Simultaneously, whenever a sentence boundary is hit in the streaming output, a TTS task is queued. The audio is fetched from Sarvam and sent back as `tts_chunk` messages so playback can start before generation finishes.
7. On the frontend, a `processTTSQueue` function plays chunks in order using an `<audio>` element or, in live mode, through the AEC AudioContext.

---

3. Live Video Chat

What it does: Full-duplex real-time chat where the bot can see the user's camera and hear them speak simultaneously.

Endpoint: `WS /api/live/ws`

Frontend (`App.jsx`): 1. Camera access is requested via `getUserMedia` . Video frames are captured every 1 second using `ImageCapture` (or canvas fallback) and sent as JPEG base64 over the WebSocket. 2. Microphone audio goes through a two-layer AEC (Acoustic Echo Cancellation) pipeline: - Layer 1: Browser's native `echoCancellation` , `noiseSuppression` , and `autoGainControl` hints. - Layer 2: An NLMS worklet (`aec-processor.js`) fed via `connectMicToAEC()` from `aecUtils.js` . TTS playback is also routed through this context so the worklet can subtract it from the mic signal. 3. PCM audio chunks are encoded and sent as `audio_chunk` WebSocket messages. 4. The frontend handles incoming message types: `text_chunk` (streams tokens into the chat bubble), `tts_chunk` (queues audio), `transcript` / `partial_transcript` (shows the user's speech), and `interrupt_ack` (handles barge-in by flushing the TTS queue).

Backend (`server.py`): 1. A `Session` object tracks per-connection state: frame buffer (last 5 frames), conversation history, world state, VAD iterator, and STT session. 2. A `ConnectionManager` with a background cleanup task removes sessions inactive for 10 minutes. 3. Audio chunks trigger VAD. When speech starts, a `StreamingSTTSession` is opened — this wraps Sarvam's synchronous streaming WebSocket in a background thread so it's compatible with asyncio. 4. When speech ends, the STT session is finalized and the transcript is sent back to the client as a `transcript` message. 5. The transcript is passed to `handle_user_question()` : - `needs_visual_context()` decides — based on keywords like "what is", "can you see", "read" — whether vision is needed. - If vision is needed and there are frames in the buffer, the latest frame is resized to 512×512 JPEG using PIL and sent to the vision LLM (GPT-4o-mini via OpenRouter) with a prompt asking it to describe the scene. A TTS filler phrase ("Let me look...") is played while waiting. - The vision description is injected into the prompt for the streaming LLM. - If no vision is needed, the last 6 messages of conversation history are used directly. 6. The LLM streams tokens. A producer/consumer queue pattern ensures TTS fetches begin as soon as each sentence ends, and audio chunks are delivered to the frontend in order. Barge-in (user speaking while bot is talking) sends an `interrupt_ack` and stops generation. 7. Scene change detection uses multi-region MD5 hashing on the base64 frame string. A change requires at least 2 of 3 regions (top, mid, bottom) to differ, avoiding false positives from small movements.

---

4. Hybrid RAG Pipeline

What it does: Retrieves the most relevant Jio FAQ answer using a combination of vector search, keyword search, and reranking.

How it's implemented (`nodes.py`):

1. **Query normalization** — A local fuzzy normalizer (`difflib.SequenceMatcher`) corrects Jio-specific product names and typos before any API call (e.g., "jioplus" → "JioPlus", "siggy" → "Swiggy").
2. **Vector encoding** — The normalized question is encoded by `SentenceTransformer('all-MiniLM-L6-v2')` into a 384-dimensional vector.
3. **Neo4j hybrid search** — A single Cypher query runs two parallel sub-queries:
4. `db.index.vector.queryNodes('faq_embeddings', 10, ...)` for semantic similarity.
5. `db.index.fulltext.queryNodes('faq_text_index', ...)` for keyword (Lucene BM25) matching.
6. Results are deduplicated (`DISTINCT`) and graph traversal adds `Topic → Subtopic → FAQ` context to each result string.
7. **Qdrant user-document search** — `search_qdrant()` from `file_workflow.py` simultaneously queries the user's uploaded PDF chunks using the same vector, filtered by `user_id` and `session_id` .
8. **CrossEncoder reranking** — All candidate strings are passed as `(question, context)` pairs to `CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')` which gives a relevance score to each. Uploaded document results get a `+5.0` score boost to ensure personal documents are always prioritized.
9. **Semantic routing** — If the best reranker score is below `-1.5` , the question is considered out-of-domain and routed to the `general_generation_node` . Otherwise the top 3 candidates go to `generate_node` .

10. **Generation** — `generate_node` constructs a system prompt with the retrieved context and invokes NVIDIA Llama 3.1 8B. `general_generation_node` uses the same model with tool-calling enabled for weather and location lookups.

---

5. PDF Document Ingestion & Retrieval

What it does: Users can upload PDFs which are then stored and searched alongside Jio FAQs.

Endpoint: `POST /api/upload`

How it's implemented (`file_workflow.py` + `server.py`):

1. The uploaded PDF is saved to a temp file and a `BackgroundTasks` job is queued so the HTTP response returns immediately.
2. `converter_pdf()` uses **Docling** to convert the PDF to Markdown, preserving structure (headings, tables, lists) for better LLM comprehension.
3. The Markdown text is split into 1000-character chunks with 200-character overlap by `RecursiveCharacterTextSplitter` from LangChain.
4. Each chunk is tagged with a `chunk_id`, `document_id`, `user_id`, and `session_id` so retrieval is always scoped to the current user and session.
5. `SentenceTransformer('all-MiniLM-L6-v2')` embeds all chunks in a single batched `encode()` call.
6. Chunks and embeddings are upserted into a Qdrant collection (`jio_documents`) with payload indexes on `user_id` and `session_id` for efficient filtered search.
7. At query time, `search_qdrant()` uses a `Filter` on both fields to retrieve only the current user's documents.

---

6. Vision (Image Upload & Analysis)

What it does: Users can upload a static image and get a description or answer from a vision LLM.

Endpoint: `POST /api/vision`

How it's implemented:

The uploaded image is base64-encoded and sent to **NVIDIA Llama 3.2 11B Vision** (`meta/llama-3.2-11b-vision-instruct`) via `langchain_nvidia_ai_endpoints`. The model returns a textual description/analysis of the image content.

In live video mode, this same approach is used dynamically — the latest camera frame from the 5-frame rolling buffer is compressed to a 512×512 JPEG and sent to **GPT-4o-mini via OpenRouter** for lower latency.

---

7. Long-Term Memory

What it does: The bot remembers personal facts about users across sessions (e.g., "User lives in Mumbai", "User prefers prepaid").

How it's implemented (`nodes.py`):

1. After every non-FAQ response, `evaluate_and_save_memory_bg()` runs as a background task.
2. It sends the (question, answer) pair to NVIDIA Llama 3.1 8B with a Memory Evaluation prompt that asks whether any personal fact worth saving was revealed.
3. The LLM replies with a structured JSON: `{"save_memory": true, "memory_content": "..."}`.
4. If `save_memory` is true, the fact is inserted into the Supabase `user_memories` table.
5. At query time, existing memories are fetched from Supabase and injected into the LLM system prompt so the bot knows about the user before they say a word.

---

8. Short-Term Memory & Conversation History

What it does: The bot maintains conversation context within a session and prevents the context window from growing indefinitely.

How it's implemented:

- **LangGraph checkpointing** — The workflow is compiled with `AsyncSqliteSaver` backed by `checkpoints.db`. Every invocation appends to the message list keyed by `thread_id` (the session ID). This gives the bot full conversation history across requests.
- **Session history API** — `GET /api/sessions/{session_id}/history` reads the LangGraph checkpoint state and reconstructs the message list.

- **Auto-summarization** — `summarize_short_term_memory_bg()` fires after each chat turn. If the session has more than 5 messages, all but the last 2 are summarized by NVIDIA Llama 3.1 8B into a single `SystemMessage`. The original messages are deleted via `RemoveMessage` and replaced with the summary, keeping the context window small.

---

9. Session Management

What it does: Each conversation is a named session that users can switch between, load, or delete.

How it's implemented:

- Sessions are stored in the Supabase `chat_sessions` table with `id`, `user_id`, `title`, and `created_at`.
- `GET /api/sessions` returns all sessions for the authenticated user, ordered by creation date.
- `DELETE /api/sessions/{session_id}` verifies ownership before deleting from Supabase.
- The frontend sidebar (`App.jsx`) lists sessions, allows switching (which loads history from the LangGraph checkpoint), starting new chats, and deleting sessions.
- Session titles are auto-generated from the first message using Groq Llama 3.1 8B (fast, low-latency).

---

10. Authentication

What it does: All API endpoints are protected. Each user sees only their own sessions and memories.

How it's implemented:

- **Frontend (`Auth.jsx`)** — Email/password login and signup using the Supabase JS client. On success, a JWT is stored in the session.
- **Backend (`server.py`)** — A `get_current_user` FastAPI dependency extracts the `Bearer` token from the `Authorization` header and calls `supabase.auth.get_user(token)` to verify it. The returned `user.id` is used as the LangGraph `thread_id`.
- The Supabase client used for data operations (sessions, memory) is always instantiated with the user's JWT so Supabase Row Level Security (RLS) policies prevent cross-user access.

---

11. A2UI Rich Cards

What it does: The LLM can respond with structured UI components embedded in the chat (e.g., a WeatherCard with city, temperature, and condition).

How it's implemented:

- `general_generation_node` instructs the LLM to return a specific JSON structure when weather data is available.
- `process_a2ui_messages()` in `server.py` parses the LLM response with a regex. If a valid `WeatherCard` JSON is found, it generates two A2UI protocol messages: `createSurface` and `updateComponents`.
- The frontend uses `@a2ui/react` and `@a2ui/web_core` to render these messages as visual cards. The `A2UICatalog.tsx` file defines the component registry.
- The spoken TTS text is cleaned to remove the JSON so the bot speaks naturally ("The weather in Mumbai is 32 degrees") while the card is displayed visually.